

Engineering with LLMs: From Prompting to Structured Systems

Reuben Brasher

March 22, 2026

Lecture 3

Failures Are Engineering Failures

Most failures with LLMs arise from lack of structure rather than model limits.

- ▶ LLMs are unreliable when used without constraints.
- ▶ Naive prompting produces inconsistent and incorrect outputs.
- ▶ Failures often result from poor workflow design.
- ▶ Writing discipline applies directly to engineering practice.
- ▶ Reliability must be deliberately engineered.
- ▶ The problem is lack of structure, not lack of intelligence.

From Writing Discipline to System Design

Disciplined writing principles translate directly into engineering workflows.

- ▶ Grounding reduces hallucination.
- ▶ Structure reduces ambiguity.
- ▶ Iteration improves correctness.
- ▶ Constraints guide output quality.
- ▶ Principles must be formalized into systems.
- ▶ Engineering replaces ad hoc prompting with processes.

Reframing the Interaction

LLM usage should be treated as a sequence of transformations.

- ▶ Naive approach relies on single prompts.
- ▶ Structured approach uses staged pipelines.
- ▶ Each stage constrains the output.
- ▶ Outputs feed into subsequent steps.
- ▶ The process becomes inspectable.
- ▶ Engineering replaces guessing with control.

The Pipeline Model

Structured pipelines decompose complex tasks into stages.

- ▶ Begin with natural language problem definition.
- ▶ Construct domain models explicitly.
- ▶ Specify schemas to constrain structure.
- ▶ Generate implementation code.
- ▶ Produce tests for validation.
- ▶ Each step increases precision.

Example Transformation Pipeline

Engineering workflows follow explicit staged transformations.

- ▶ Natural language becomes domain concepts.
- ▶ Domain concepts become structured schemas.
- ▶ Schemas become typed models.
- ▶ Models become implementation code.
- ▶ Code becomes validated through tests.
- ▶ Each stage is reviewable and explicit.

Conversational Grounding as First Step

Engineering workflows must begin with explicit grounding.

- ▶ Explain the domain before generating outputs.
- ▶ Require restatement of key concepts.
- ▶ Validate terminology and definitions.
- ▶ Avoid premature implementation.
- ▶ Establish shared understanding.
- ▶ Grounding constrains the solution space.

Why Grounding Matters

Correct outputs depend on correct conceptual alignment.

- ▶ Misunderstood domains lead to incorrect outputs.
- ▶ Early errors propagate through pipelines.
- ▶ Grounding prevents compounding mistakes.
- ▶ Alignment reduces downstream corrections.
- ▶ Early validation is low-cost.
- ▶ Correctness begins before coding.

Structured Representations

Engineering requires moving from ambiguity to precision.

- ▶ Natural language is inherently ambiguous.
- ▶ Tables introduce structured organization.
- ▶ JSON enforces explicit fields.
- ▶ Typed models enforce constraints.
- ▶ Each step reduces interpretive freedom.
- ▶ Precision improves reliability.

Representation Progression

A sequence of representations improves clarity.

- ▶ Natural language is flexible but vague.
- ▶ Tables provide structured readability.
- ▶ JSON enables machine-readable precision.
- ▶ Typed models enforce validation rules.
- ▶ Each stage eliminates ambiguity.
- ▶ Transformation is the core strength.

Transformations Over Generation

Reliability comes from transforming structured data.

- ▶ Unconstrained generation introduces variability.
- ▶ Transformation preserves structure.
- ▶ Structured inputs reduce hallucination.
- ▶ Intermediate steps enable validation.
- ▶ Each stage can be tested.
- ▶ Reliability emerges from transitions.

Test-Driven Development with LLMs

Tests should constrain system behavior early.

- ▶ Generate tests alongside implementation.
- ▶ Tests define expected behavior.
- ▶ Tests constrain future outputs.
- ▶ Implementation follows specifications.
- ▶ Early validation reduces errors.
- ▶ Tests clarify vague requirements.

Risks in LLM-Generated Tests

Generated tests can reinforce incorrect assumptions.

- ▶ Models may validate their own mistakes.
- ▶ Tests can encode flawed logic.
- ▶ Passing tests can create false confidence.
- ▶ Human review is required.
- ▶ Tests must reflect real requirements.
- ▶ Validation must precede trust.

Human Ownership of Tests

Tests must be treated as authoritative human specifications.

- ▶ Review all generated tests carefully.
- ▶ Ensure alignment with intended behavior.
- ▶ Reject incorrect assumptions.
- ▶ Modify tests before use.
- ▶ Treat tests as ground truth.
- ▶ Do not outsource correctness.

Role Separation in Workflows

Different engineering roles must be separated.

- ▶ Architect defines structure.
- ▶ Coder implements specifications.
- ▶ Reviewer evaluates outputs.
- ▶ Roles operate independently.
- ▶ Avoid mixing responsibilities.
- ▶ Separation reduces errors.

Separation of Concerns

Classical engineering principles apply directly to LLM workflows.

- ▶ Do not combine design and implementation.
- ▶ Maintain clear stage boundaries.
- ▶ Use separate interactions for roles.
- ▶ Avoid reinforcing feedback loops.
- ▶ Preserve independent evaluation.
- ▶ Structure reduces confusion.

Managing Context and Drift

Long interactions degrade output quality.

- ▶ Models inherit earlier mistakes.
- ▶ Context accumulates noise.
- ▶ Drift causes inconsistency.
- ▶ Long sessions reduce reliability.
- ▶ Errors become hidden.
- ▶ Context requires management.

Practices for Controlling Context

Context must be explicitly controlled in workflows.

- ▶ Restart conversations frequently.
- ▶ Isolate tasks into sessions.
- ▶ Avoid long histories.
- ▶ Provide only relevant information.
- ▶ Remove outdated context.
- ▶ Treat context as input.

Context as a Liability

Uncontrolled context reduces reliability.

- ▶ More context does not improve results.
- ▶ Irrelevant data causes confusion.
- ▶ Early mistakes propagate forward.
- ▶ Models cannot filter noise reliably.
- ▶ Precision requires selective input.
- ▶ Constraints improve performance.

Multi-Model Workflows

Different models can serve different roles.

- ▶ One model generates outputs.
- ▶ Another evaluates outputs.
- ▶ Models specialize in tasks.
- ▶ Combining models increases robustness.
- ▶ Cross-checking reduces errors.
- ▶ Avoid reliance on a single model.

Models as Specialized Tools

LLMs should be treated as tools with strengths and limits.

- ▶ Some models excel at generation.
- ▶ Others excel at critique.
- ▶ Selection depends on task.
- ▶ Combine complementary capabilities.
- ▶ Tool composition improves results.
- ▶ Orchestration replaces reliance.

Structured System Design in Practice

Structured engineering principles appear in real systems.

- ▶ Domain modeling precedes implementation.
- ▶ Commands define system behavior.
- ▶ Schemas replace ambiguity.
- ▶ Deterministic workflows ensure reproducibility.
- ▶ Context is constructed explicitly.
- ▶ Systems operate on structured transformations.

Bounded Contexts and Separation

Clear system boundaries improve reliability.

- ▶ Separate domains for key functions.
- ▶ Avoid shared mutable state.
- ▶ Use explicit interfaces.
- ▶ Orchestrate at higher layers.
- ▶ Independence improves clarity.
- ▶ Structure prevents coupling.

Discussion

Engineering with LLMs requires structured system design.

- ▶ Reliability comes from staged transformations.
- ▶ Grounding reduces ambiguity.
- ▶ Tests enforce correctness.
- ▶ Role separation improves clarity.
- ▶ Context must be controlled.
- ▶ Systems outperform prompts.

Final Remarks

Engineering introduces structure but systems must operate over time.

- ▶ How do workflows scale in practice.
- ▶ How do systems evolve with use.
- ▶ What happens under long-term operation.
- ▶ How is state maintained across interactions.
- ▶ How do errors accumulate over time.
- ▶ Next step is systems over time.

References I